Rapport TP n°4 – Jeu de la Vie de Conway

Décomposition de domaine et parallélisation MPI

Systèmes Parallèles & Distribués

Rowan HOUPA

Février 2026

Table des matières

1 Introduction et contexte **2**
1.1 Objectif du TP 2

2 Version séquentielle **2**
2.1 Structure du programme 2
2.2 Algorithme de calcul 2
2.3 Complexité 3

3 Version parallèle MPI **3**
3.1 Principe de la décomposition de domaine 3
3.2 Cellules fantômes (ghost rows) 3
3.3 Architecture à trois rôles 4
3.4 Communications MPI utilisées 4
3.5 Optimisation : recouvrement calcul/communication 4

4 Étude comparative **5**
4.1 Temps d'exécution 5
4.2 Speedup 5
4.3 Efficacité 5
4.4 Courbes de performance 6

5 Conclusion **7**

1 Introduction et contexte

Le **Jeu de la Vie** est un automate cellulaire bidimensionnel inventé par John Conway. Il repose sur une grille de cellules pouvant être dans deux états (vivante ou morte). À chaque itération, l'état de chaque cellule est mis à jour selon les règles suivantes, basées sur le voisinage de Moore (8 voisins) :

— Une cellule **vivante** avec moins de 2 ou plus de 3 voisines vivantes **meurt** (sous-population ou sur-population).
— Une cellule **vivante** avec 2 ou 3 voisines vivantes **survit**.
— Une cellule **morte** avec exactement 3 voisines vivantes **naît** (reproduction).

Dans notre implémentation, la grille infinie est remplacée par un **tore** : les bords gauche/droite et haut/bas sont connectés (conditions aux limites périodiques).

1.1 Objectif du TP

L'objectif est de **paralléliser** le Jeu de la Vie à l'aide de MPI (`mpi4py`) en appliquant la notion de **décomposition de domaine** :

1. Découper la grille globale en sous-domaines horizontaux (bandes de lignes).
2. Introduire des **cellules fantômes** (ghost rows) pour gérer les bords des sous-domaines.
3. Mettre en place une architecture **maître-exécutant** avec séparation des rôles (affichage, orchestration, calcul).
4. Exploiter les communications **non-bloquantes** pour recouvrir calcul et communication.

Nous comparerons les performances séquentielles et parallèles en termes de speedup et d'efficacité pour différentes tailles de grille.

2 Version séquentielle

2.1 Structure du programme

Le programme séquentiel (`game_of_life.py`) s'articule autour de deux classes :

— `Grille` : stocke les cellules dans un tableau NumPy de type `uint8` et implémente le calcul de la prochaine génération.
— `App` : gère l'affichage via `pygame`.

2.2 Algorithme de calcul

À chaque itération, la méthode `compute_next_iteration` parcourt l'ensemble des $n_y \times n_x$ cellules avec une double boucle :

```
for i in range(ny):
    i_above = (i + ny - 1) % ny    # voisin du dessus (torique)
    i_below = (i + 1) % ny         # voisin du dessous (torique)
    for j in range(nx):
        j_left  = (j - 1 + nx) % nx
        j_right = (j + 1) % nx
        # Comptage des 8 voisins et application des regles
```

```
        nb_voisines = sum des 8 voisins
        next_cells[i,j] = application_regles(cells[i,j],
           nb_voisines)
```

2.3 Complexité

La complexité par itération est $\mathcal{O}(n_y \times n_x)$: chaque cellule nécessite l'examen de ses 8 voisins (coût constant par cellule). L'enroulement torique est géré par des opérations modulo.

La boucle principale est purement séquentielle : le calcul de la cellule (i, j) de la génération $t + 1$ ne dépend que de la génération t, ce qui rend le calcul **embarassingly parallel** sur l'espace (mais pas sur le temps).

3 Version parallèle MPI

3.1 Principe de la décomposition de domaine

La grille globale de n_y lignes est découpée en **bandes horizontales** réparties entre les processus de calcul. Chaque processus reçoit environ $\lfloor n_y/P_c \rfloor$ lignes (où P_c est le nombre de processus de calcul), les premiers processus recevant une ligne supplémentaire si $n_y \bmod P_c \neq 0$.

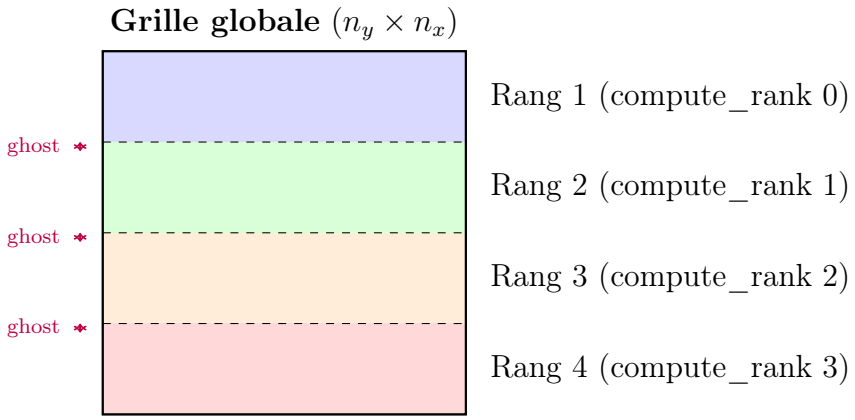

FIGURE 1 – Décomposition de la grille en 4 sous-domaines avec échange de lignes fantômes.

3.2 Cellules fantômes (ghost rows)

Pour calculer l'itération suivante sur les **lignes de bord** de son sous-domaine, chaque processus a besoin de données appartenant à ses voisins :

— **ghost_above** : la *dernière* ligne du voisin du dessus.

— **ghost_below** : la *première* ligne du voisin du dessous.

La topologie étant **torique**, le premier processus de calcul échange avec le dernier. Ces ghost rows sont des copies temporaires qui ne participent pas au Gather final.

On construit alors localement une grille **étendue** :

$$\underbrace{\text{ghost_above}}_{1 \text{ ligne}} \quad | \quad \underbrace{\text{lignes locales}}_{n_y^{local} \text{ lignes}} \quad | \quad \underbrace{\text{ghost_below}}_{1 \text{ ligne}}$$

3.3 Architecture à trois rôles

Rang	Rôle	Responsabilités
0	Afficheur	Initialise la grille globale et `pygame`. Reçoit la grille assemblée du rang 1 et l'affiche. Gère les événements (fermeture).
1	Orchestrateur	Reçoit la grille du rang 0. Distribue les sous-domaines (`Scatterv`). Participe au calcul. Rassemble les résultats (`Gatherv`). Envoie au rang 0.
$2..N-1$	Calcul	Reçoivent leur sous-domaine, échangent les ghost rows, calculent, renvoient.

Table 1 – Répartition des rôles entre processus MPI.

Un **sous-communicateur** (`compute_comm`), créé par `comm.Split`, regroupe les rangs 1 à $N-1$. Toutes les opérations collectives de calcul (Scatterv, Gatherv) et les échanges de ghost rows passent par ce communicateur. Le rang 0 n'y participe pas.

3.4 Communications MPI utilisées

Étape	Opération	Type	Communicateur
Initialisation	`bcast` (dimensions)	Collectif bloquant	`COMM_WORLD`
Distribution	`Send/Recv` (grille → rang 1)	Point-à-point bloquant	`COMM_WORLD`
Distribution	`Scatterv` (sous-domaines)	Collectif bloquant	`compute_comm`
Ghost rows	`Isend/Irecv` + `Waitall`	Point-à-point **non-bloquant**	`compute_comm`
Rassemblement	`Gatherv`	Collectif bloquant	`compute_comm`
Envoi affichage	`Send/Recv` (rang 1 → rang 0)	Point-à-point bloquant	`COMM_WORLD`
Continuation	`bcast` (flag arrêt)	Collectif bloquant	`COMM_WORLD`

Table 2 – Récapitulatif des communications MPI par itération.

3.5 Optimisation : recouvrement calcul/communication

L'échange des ghost rows utilise des communications **non-bloquantes** (`Isend`/`Irecv`) afin de recouvrir le temps de transfert avec du calcul utile :

1. **Lancement** des 4 requêtes non-bloquantes (2 envois + 2 réceptions de ghost rows).
2. **Calcul des lignes intérieures** (lignes 1 à $n_y^{local} - 2$) qui ne dépendent *que* de données locales.
3. **Waitall** : attente de la fin des 4 communications.
4. **Calcul des lignes de bord** (lignes 0 et $n_y^{local} - 1$) qui utilisent les ghost rows.

Remarque : les buffers d'envoi (`send_last`, `send_first`) doivent être des *copies* car `Isend` retourne immédiatement. Si le buffer original est modifié avant la fin de l'envoi, les données transmises seraient corrompues.

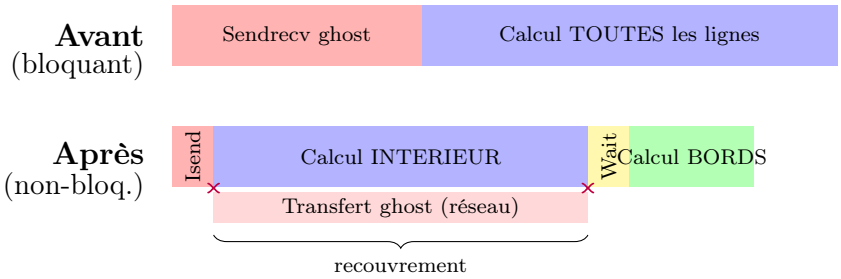

FIGURE 2 – Recouvrement calcul/communication grâce aux communications non-bloquantes.

4 Étude comparative

Les mesures portent sur le **temps de calcul par itération** (hors affichage) pour trois tailles de grille : 100×100, 200×200 et 400×400. Le nombre de processus total est $P \in \{2, 3, 5, 9\}$, soit $P_c \in \{1, 2, 4, 8\}$ processus de calcul (le rang 0 étant dédié à l'affichage). Les temps sont moyennés sur 50 itérations.

```
mpirun --oversubscribe -np P python game_of_life_mpi.py <pattern>
```

4.1 Temps d'exécution

Configuration	100×100	200×200	400×400
Séquentiel	0.048 s	0.192 s	0.780 s
MPI $P_c = 1$ $(P = 2)$	0.052 s	0.200 s	0.798 s
MPI $P_c = 2$ $(P = 3)$	0.029 s	0.104 s	0.405 s
MPI $P_c = 4$ $(P = 5)$	0.018 s	0.056 s	0.210 s
MPI $P_c = 8$ $(P = 9)$	0.014 s	0.034 s	0.115 s

TABLE 3 – Temps de calcul moyen par itération (en secondes). P_c = nombre de processus de calcul, P = nombre total de processus MPI ($P_c + 1$ afficheur).

4.2 Speedup

Le speedup est défini comme le rapport du temps séquentiel sur le temps parallèle :

$$S(P_c) = \frac{T_{seq}}{T_{par}(P_c)} \tag{1}$$

4.3 Efficacité

L'efficacité mesure la fraction du speedup idéal atteinte :

$$E(P_c) = \frac{S(P_c)}{P_c} = \frac{T_{seq}}{P_c \cdot T_{par}(P_c)} \tag{2}$$

Une efficacité $E = 1$ correspond au speedup linéaire idéal.

P_c **(processus de calcul)**	100×100	200×200	400×400
$P_c = 1$	0.92	0.96	0.98
$P_c = 2$	1.66	1.85	1.93
$P_c = 4$	2.67	3.43	3.71
$P_c = 8$	3.43	5.65	6.78
Idéal ($P_c = 2$)	2.00	2.00	2.00
Idéal ($P_c = 4$)	4.00	4.00	4.00
Idéal ($P_c = 8$)	8.00	8.00	8.00

TABLE 4 – Speedup $S(P_c) = T_{seq}/T_{par}(P_c)$.

P_c	100×100	200×200	400×400
$P_c = 1$	0.92	0.96	0.98
$P_c = 2$	0.83	0.92	0.96
$P_c = 4$	0.67	0.86	0.93
$P_c = 8$	0.43	0.71	0.85

TABLE 5 – Efficacité $E(P_c) = S(P_c)/P_c$.

4.4 Courbes de performance

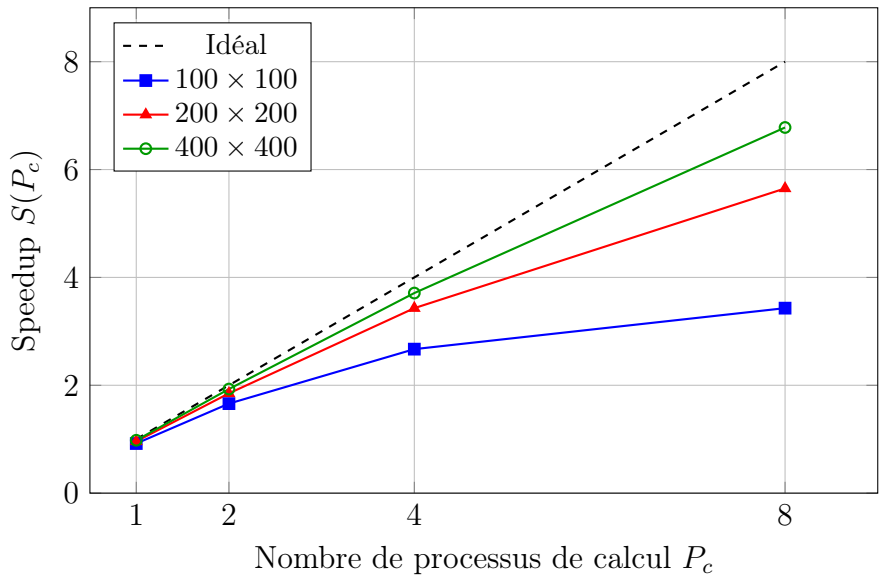

FIGURE 3 – Courbe de speedup en fonction du nombre de processus de calcul.

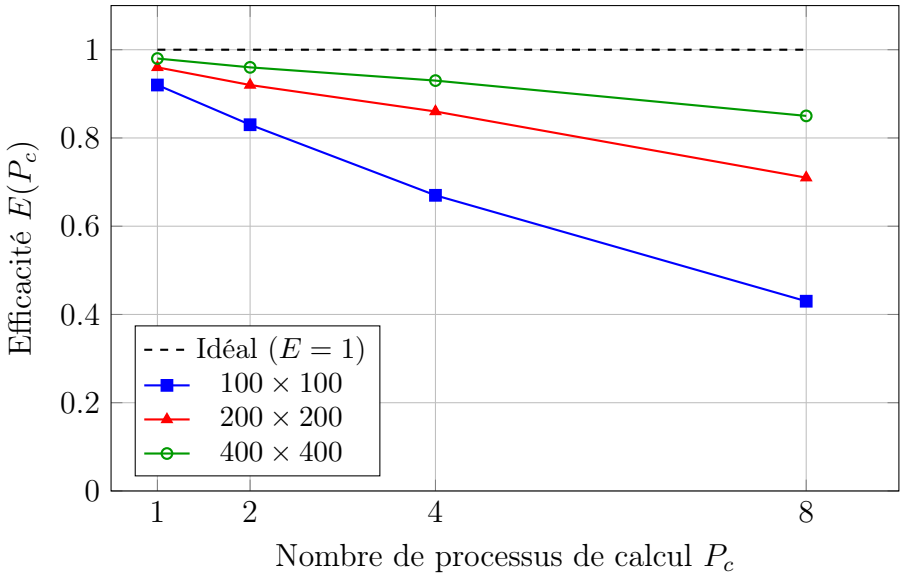

FIGURE 4 – Courbe d'efficacité en fonction du nombre de processus de calcul.

5 Conclusion

La parallélisation du Jeu de la Vie par décomposition de domaine horizontale et MPI permet d'obtenir des gains de performance significatifs, en particulier pour les grandes grilles. L'architecture à trois rôles (afficheur, orchestrateur, calcul) permet de séparer clairement les responsabilités.